TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

BE | Year III / Part II | CT 655

EMBEDDED SYSTEM — QUESTION BANK WITH ANSWERS

GAP TOPICS SUPPLEMENT

Important Questions Not Covered in Chapter Files CH1–CH10

Chapters Covered in This File:

- Ch.1 — Custom Processor Design: Fibonacci, Mean, x^n, Sum of Digits, Largest N, Prime, Optimization
- Ch.2 — Processor Design: Pipelining (5/6 stage), DSP, ASIP, Control Unit Sub-ops, VLIW, Criteria
- Ch.3 — Software Development: Dev Process, Debugger/Simulator/Emulator, ISA Bus Protocols
- Ch.4 — Memory: ROM Design (truth table), ROM Composition, Cache Replacement, EEPROM, SRAM vs DRAM
- Ch.5 — Interfacing: Port-based vs Bus-based I/O, Fixed ISR, Interface Circuit Design, Strobe/Handshake
- Ch.6 — RTOS: Mutual Exclusion, Busy-Wait, Multithreading, User/Kernel Threads, 6-Process Scheduling
- Ch.7 — Control System: Practical Issues, Computer Control Benefits, PD/PI Controllers
- Ch.8 — IC Technology: CMOS Layout (F=xz+yz'), Semi-custom ASIC Types
- Ch.9 — 8051: 7-Seg CA (9→0), Switch→LED, Min Config, 75% Duty Cycle, 2-Switch Blink, Down Counter
- Ch.10 — VHDL: PROCESS/Dataflow, 1:8 DeMUX, Decoder, Elevator FSM, BCD Counter, Seq Detector 1101

Full Marks: 80 | Pass Marks: 32 | Time: 3 hrs

Chapter 1 Gap Topics: Custom Processor Design

G1.1 Design a single-purpose processor that outputs Fibonacci numbers up to n places. Show algorithm, FSMD, state diagram, datapath, and optimization opportunities.	**2080 Ashwin (Back) | 2075 Baisakh (Back) | 2075 Bhadra (Back) | 2076 Baisakh (Regular) [5+3]**

ANSWER

Algorithm for Fibonacci Sequence:

1. Initialize: a=0, b=1, count=n.
2. If count=0, output nothing. If count≥1, output a (=0).
3. While count > 1: output b; compute next = a+b; a=b; b=next; count=count−1.
4. Done when count=0.

FSMD (Finite State Machine with Datapath):

```
States: IDLE → INIT → CHECK → OUTPUT → UPDATE → DONE

State diagram:
  IDLE ──(go=1)──▶ INIT ──────▶ CHECK ──(count=0)──▶ DONE
                  (a=0,b=1,      │                   (done=1)
                   cnt=n)        │ count>0
                                 ▼
                            OUTPUT ──▶ UPDATE ──▶ CHECK
                           (out=a)    (next=a+b
                                       a=b
                                       b=next
                                       cnt=cnt-1)
```

Datapath Design:

```
Registers: Ra (8-bit), Rb (8-bit), Rcount (8-bit), Rnext (8-bit)
Functional Units:
  Adder:   Rnext ← Ra + Rb
  Counter: Rcount ← Rcount - 1
  MUX1:    Ra ← 0 (INIT) | Rb (UPDATE)
  MUX2:    Rb ← 1 (INIT) | Rnext (UPDATE)

Datapath block:
  go ──▶ IDLE/INIT: Ra←0, Rb←1, Rcount←n
         CHECK: compare Rcount vs 0 → zero_flag
         OUTPUT: output ← Ra
         UPDATE: Rnext←Ra+Rb; Ra←Rb; Rb←Rnext; Rcount←Rcount-1

Controller outputs: ld_a, ld_b, ld_next, ld_count, out_en, done
```

Optimization Opportunities:

- State merging: Combine OUTPUT and UPDATE into a single state to save one clock cycle per iteration.
- Resource sharing: The adder used for Rnext=Ra+Rb can also serve the decrement (Rcount−1) by adding −1 (or sharing ALU).
- Unrolling: For small fixed n, unroll the loop to a purely combinational datapath (no counter needed).
- Pipelining: For large n, pipeline the add and register-update stages to overlap consecutive iterations.

G1.2 Design a custom single-purpose processor to compute x^n (x raised to power n). Show algorithm, FSMD, datapath, and controller block diagram.	**2080 Chaitra (Back) | 2074 Bhadra (Regular) | 2078 Chaitra (Back) [8]**

ANSWER

Algorithm for x^n (repeated multiplication):

1. Input: x (base), n (exponent). Initialize result=1, count=n.
2. While count > 0: result = result × x; count = count − 1.
3. Output result = x^n.

FSMD State Diagram:

```
  IDLE ──(go=1)──▶ INIT ──▶ CHECK ──(count=0)──▶ DONE
                  (res=1,     │                   (result<=Rres, done=1)
                   cnt=n)     │ count>0
                              ▼
                         MULTIPLY ──▶ DECREMENT ──▶ CHECK
                         (Rres=        (Rcnt=
                          Rres*x)       Rcnt-1)
```

Datapath:

```
Registers:  Rres (result, 16-bit), Rcnt (counter, 8-bit)
Inputs:     x (8-bit), n (8-bit)
Units:      Multiplier: Rres ← Rres × x
            Decrementer: Rcnt ← Rcnt − 1
            Comparator: zero_flag ← (Rcnt = 0)

Controller signals:
  ld_res  — load Rres←1 (INIT) or Rres←Rres×x (MULTIPLY)
  ld_cnt  — load Rcnt←n (INIT) or Rcnt←Rcnt-1 (DECREMENT)
  done    — asserted in DONE state
  out_en  — output enable in DONE state
```

Controller Block Diagram:

```
Inputs: clk, reset, go, zero_flag (from datapath comparator)
Outputs: ld_res, ld_cnt, sel_init, done


┌──────────────────────────────────────────┐
│           CONTROLLER (FSM)               │
│                                          │
│  go ──► state register ──► next-state    │
│  zero_flag ──►  logic  ──► output logic  │
│                    │                     │
│  ld_res, ld_cnt, sel_init, done ─────────┼──► Datapath
└──────────────────────────────────────────┘

zero_flag, done ◄──────────────────────────── Datapath
```

G1.3 What is optimization? Explain the optimization opportunities available for custom single-purpose processors with suitable examples.

2079 Jestha (Back) | 2072 Magh (Back) | 2080 Ashwin (Back) | 2071 Bhadra (Regular) [8]

ANSWER

Optimization of Custom Single-Purpose Processors:

Optimization is the process of improving a custom processor design to meet design metrics (speed, area, power, cost) more effectively without changing the functional behavior.

Key Optimization Opportunities:

1. State Merging: Combine multiple FSM states that can execute in the same clock cycle. Example: If OUTPUT and UPDATE states each take 1 cycle but have no data dependencies, merge into one state → saves 1 clock cycle per iteration.

2. Resource Sharing: Share functional units (adders, multipliers) across states that don't execute simultaneously. Example: In GCD processor, the subtractor used for Ra←Ra−Rb can also serve Rb←Rb−Ra by using a MUX at the input → saves one hardware subtractor.

3. Pipelining the Datapath: Insert pipeline registers to overlap execution of multiple iterations. If multiply takes 3ns, pipeline at 1ns intervals → throughput improves 3×. Latency increases but throughput improves.

4. Loop Unrolling: For processors with a fixed small loop count, replace the loop with direct computation. Example: For n=4 fixed: result = x*x*x*x directly, no counter needed → eliminates comparator, counter, FSM overhead.

5. Operator Strength Reduction: Replace expensive operations with cheaper ones. Example: Replace x^2 with x*x (saved), replace multiply-by-2 with left shift.

6. Concurrent Execution: Execute independent operations in the same clock cycle using parallel hardware. Example: In Fibonacci, compute next=a+b AND output current value in same cycle.

7. Custom Word Width: Use only as many bits as required for the computation. Example: If maximum intermediate value fits in 12 bits, use 12-bit adder instead of 16-bit → smaller, faster, lower power.

Technique	Goal	Trade-off
State Merging	Reduce clock cycles (latency)	More complex combinational logic per state
Resource Sharing	Reduce hardware area (cost)	May increase cycle time or add MUX delay
Pipelining	Increase throughput	Increased latency; pipeline hazard management
Loop Unrolling	Eliminate loop overhead	Larger circuit area for fixed n only
Concurrent Execution	Reduce latency	More parallel hardware units needed

Chapter 2 Gap Topics: Processor Design

G2.1 What is pipelining? Explain 5-stage pipeline concept with a diagram. Describe features of a DSP (Digital Signal Processor).	2078 Chaitra (Back) | 2075 Baisakh (Back) | 2073 Magh (Back) [1+3+4]

ANSWER

Pipelining:

Pipelining is a technique of decomposing the instruction execution cycle into multiple sequential stages, each handled by a dedicated hardware unit, so that multiple instructions can be processed concurrently — each in a different stage. Like an assembly line in a factory.

5-Stage Pipeline (Classic RISC):

```
Stage 1: IF  — Instruction Fetch: PC→Memory, read instruction, PC←PC+4
Stage 2: ID  — Instruction Decode: Decode opcode, read register file
Stage 3: EX  — Execute: ALU performs operation (add/sub/compare/shift)
Stage 4: MEM — Memory Access: Read/write data memory (for load/store)
Stage 5: WB  — Write Back: Write result to register file

Timing diagram (5 instructions, 5 cycles to fill pipeline):
Cycle:      1    2    3    4    5    6    7    8    9
Instr I1:  IF   ID   EX  MEM   WB
Instr I2:       IF   ID   EX  MEM   WB
Instr I3:            IF   ID   EX  MEM   WB
Instr I4:                 IF   ID   EX  MEM   WB
Instr I5:                      IF   ID   EX  MEM   WB

After filling (steady state): 1 instruction completes every clock cycle
Speedup: Up to N× for N-stage pipeline (limited by hazards)

Hazards (performance limiters):
  Data hazard: Instr needs result of previous instr still in pipeline
    → Solution: Forwarding/bypassing; stall (bubble insertion)
  Control hazard: Branch outcome not known until EX stage
    → Solution: Branch prediction; branch delay slot
  Structural hazard: Two instr need same hardware unit same cycle
    → Solution: Duplicate hardware (Harvard architecture separates I/D
memory)
```

DSP (Digital Signal Processor) Features:

- Single-cycle MAC (Multiply-Accumulate): Core DSP operation y+=x*h computed in one clock cycle. Essential for FIR/IIR filters, FFT, convolution.

- Hardware loop controller: Zero-overhead loops — loop count checked without branch instruction overhead.
- Multiple memory buses: Separate data buses for simultaneous fetch of two data operands (Harvard dual-bus) — needed for MAC y=A[i]*B[i].
- Special addressing modes: Circular buffer (modulo addressing) for delay lines; bit-reversed addressing for FFT butterfly.
- Saturating arithmetic: On overflow, clamps to MAX/MIN instead of wrapping — critical for audio/signal processing to avoid distortion.
- Dedicated fixed-point units: Optimized for Q-format fixed-point (1.15 or 1.31) arithmetic common in signal processing.
- Examples: TI TMS320 series, Analog Devices SHARC, ARM Cortex-M4 (with DSP extensions).

G2.2 Explain the sub-operations of the control unit of a general-purpose processor with necessary illustration. How does ASIP serve as a compromise between GPP and single-purpose processors?	**2081 Chaitra (Regular) | 2073 Bhadra (Back) | 2078 Poush (Back) [2+6]**

ANSWER

General-Purpose Processor — Block Diagram:

```
┌──────────────────────────────────────────────┐
│          General-Purpose Processor           │
│  ┌──────────┐          ┌──────────────────┐  │
│  │ Program  │ PC──────►│  Instruction     │  │
│  │ Counter  │          │  Memory (ROM)    │  │
│  └──────────┘          └──────────────────┘  │
│                                │ Instruction │
│  ┌──────────────────────────────▼──────────┐  │
│  │          CONTROLLER                     │  │
│  │ IR (Instruction Register)               │  │
│  │ Decoder → Control Signals               │  │
│  └─────────────────────────────────────────┘  │
│      Control signals   │                      │
│  ┌──────────────────────▼──────────────────┐  │
│  │          DATAPATH                       │  │
│  │ Regs (R0-R31) ◄──► ALU ◄──► MUXes       │  │
│  │ MAR, MDR, ACC ◄──► Data Memory (RAM)    │  │
│  └─────────────────────────────────────────┘  │
└──────────────────────────────────────────────┘
```

Controller Sub-Operations (for a typical instruction cycle):

1. Fetch (FE): PC→MAR; Memory[MAR]→MDR; MDR→IR; PC←PC+4. Fetches next instruction from program memory into Instruction Register.

2. Decode (DE): IR decoded by instruction decoder. Control unit reads opcode field, determines instruction type (R-type, load, store, branch). Reads source register operands from register file.
3. Register File Read (RR): Register numbers from IR fields sent to register file. Register file outputs two operand values to ALU input latches.
4. ALU Execute (ALU): ALU performs operation specified by decoded opcode (add/sub/AND/OR/compare/shift). Result stored in ALU output register.
5. Memory Access (MEM): For load: MAR←ALU_result; MDR←Memory[MAR]. For store: MAR←address; Memory[MAR]←register value. Skipped for R-type instructions.
6. Write Back (WB): ALU result or loaded data written back to destination register in register file. PC updated (branch/jump if needed).

ASIP — Application Specific Instruction Set Processor:

An ASIP is a processor with a custom instruction set designed for a specific application domain — a compromise between a general-purpose processor (flexible but large/slow/power-hungry) and a single-purpose processor (optimal but inflexible).

Property	Single-Purpose Processor	ASIP	General-Purpose Processor
Flexibility	None (one algorithm only)	Medium (domain-specific)	Maximum (any program)
Performance	Highest for specific task	High for target domain	Lower (general overhead)
Power	Lowest	Low-Medium	Highest
Programmability	None	Yes (custom ISA)	Yes (standard ISA)
NRE Cost	High (full custom)	Medium (custom ISA)	Low (existing processor)
Examples	GCD chip, JPEG encoder	DSP, GPU, crypto processor	ARM, x86, RISC-V

- ASIP adds custom instructions to a base RISC core for the target domain — e.g., a MAC instruction for DSP, AES instruction for encryption, JPEG-specific operations for image processing.

Chapter 3 Gap Topics: Software Design

G3.1 Explain the software development process for embedded systems. Define Debugger, Simulator, and Emulator. Describe the implementation and verification phases.	2082 Kartik | 2081 Chaitra| 2080 Chaitra | 2081 Ashwin (Back) [4+4]

ANSWER

Embedded Software Development Process:

```
Requirements → Architecture Design → Component Design → Coding
    → Integration → Testing/Verification → Deployment → Maintenance
Embedded-specific flow:
  1. Requirements analysis: functional + timing + power constraints
  2. Hardware/Software partitioning: decide what runs in SW vs HW
  3. Algorithm design: flowcharts, pseudocode, state machines
  4. Cross-compilation: compile on host PC targeting embedded CPU
     (Host: x86 PC; Target: ARM/8051/DSP)
  5. Simulation: run on software simulator before hardware available
  6. Debugging: use ICE, JTAG, printf, logic analyzer
  7. Integration testing: test SW with actual hardware
  8. Performance profiling: measure timing, identify bottlenecks
```

Implementation Phase:

- Writing code in C, C++, or assembly targeting the embedded processor.
- Cross-compiler (e.g., arm-gcc, SDCC for 8051) compiles source to target machine code.
- Linker maps compiled objects to target memory addresses (ROM for code, RAM for data/stack).
- Locator: assigns absolute memory addresses; generates .hex or .bin file for flashing.

Verification Phase:

- Unit testing: test each function/module independently with test vectors.
- Integration testing: test all modules together — check interfaces, timing.
- Hardware-in-the-loop (HIL): test embedded SW with actual hardware peripherals.
- Regression testing: re-run all tests after any code change.

Tool	Definition	Use Case
Debugger	Software tool that allows stepping through code, setting breakpoints, inspecting registers and memory at runtime	Finding logic bugs, stepping through ISRs, checking variable values
Simulator	Software that models the target processor and memory on a host PC — executes target code without real hardware	Early testing before hardware available; functional verification; timing estimation
Emulator (ICE)	In-Circuit Emulator — replaces target processor with a device that runs target code at full speed with real-time debugging visibility	Testing on real hardware; timing-critical debugging; hardware interface testing

Chapter 4 Gap Topics: Memory

G4.1 Compose a 4K×32 ROM using 2K×16 ROM chips. Show the chip select and address connection diagram.	**2081 Chaitra (Regular) | 2082 Kartik (Back) | 2079 Chaitra (Regular) [4+4]**

ANSWER

Given: Build 4K×32 ROM using 2K×16 ROM chips.

```
Target ROM: 4K × 32
  • 4K locations = 4096 = 2¹² → 12 address bits (A0–A11)
  • 32 data bits wide

Available chip: 2K × 16
  • 2K locations = 2048 = 2¹¹ → 11 address bits (A0–A10)
  • 16 data bits wide

Chips needed:
  • Word-width expansion (16→32 bits): 32/16 = 2 chips in PARALLEL
    (connected same address, data buses concatenated)
  • Address space expansion (2K→4K): 4K/2K = 2 banks in SERIES
    (use A11 to select which bank)
  • Total chips = 2 (width) × 2 (depth) = 4 chips

Connection diagram:

Address bus A11-A0 (12 bits):
  A10-A0 (11 bits) ──────────────────────────────────────────────────► all 4
chips
  A11 ──► Decoder/A11 select:
          A11=0 → CS of Bank 0 chips (Chip1 + Chip2) active
          A11=1 → CS of Bank 1 chips (Chip3 + Chip4) active

Data bus D31-D0 (32 bits):
  Bank 0:
    Chip1 (2K×16): address A10-A0, D15-D0  (lower 16 bits)
    Chip2 (2K×16): address A10-A0, D31-D16 (upper 16 bits)
  Bank 1:
    Chip3 (2K×16): address A10-A0, D15-D0
    Chip4 (2K×16): address A10-A0, D31-D16

  Address 0000H-07FFH → A11=0 → Bank 0 (Chip1+Chip2) selected
  Address 0800H-0FFFH → A11=1 → Bank 1 (Chip3+Chip4) selected
```

G4.2 Design a ROM to store the given truth table information. Show the internal ROM structure with address decoder and storage cells.	**2080 Chaitra (Back) | 2078 Poush (Back) | 2075 Bhadra (Back) | 2074 Bhadra (Regular) [3+5]**

ANSWER

Generic ROM Design from Truth Table:

Given truth table (example from 2080 Chaitra exam — 4 inputs x,y → 4 outputs F1,F2,F3,F4):

Address	x	y	F1	F2	F3	F4	Stored Word
0 (00)	0	0	1	0	0	0	1000 = 8H
1 (01)	0	1	0	1	1	1	0111 = 7H
2 (10)	1	0	0	0	1	1	0011 = 3H (example)
3 (11)	1	1	1	0	0	1	1001 = 9H (example)

ROM Internal Structure (for n-address, m-data ROM):

```
    Address inputs                          Data outputs
    A[n-1:0] ──▶ ┌──────────────────┐ ──▶ D[m-1:0]
                 │  n-to-2ⁿ Decoder │
                 │  (address decode) │
                 └──────────┬───────┘
                            │ 2ⁿ word lines (one per address)
                            ▼
                ┌──────────────────────────────┐
                │  Storage Array (2ⁿ × m)       │
                │  Each row = one stored word   │
                │  Each column = one bit line   │
                │                               │
                │  Programmed fuse/anti-fuse    │
                │  or mask ROM implant          │
                └──────────────────────────────┘
               m bit lines → output buffers → D[m-1:0]

Design steps:
  1. Determine address bits: n = ceil(log2(number_of_rows))
  2. Determine data bits: m = number_of_output_functions
  3. Fill storage array: ROM[i] = {F1,F2,...,Fm} for address i
  4. Mark cell as '1' (intact fuse) or '0' (blown fuse) per truth table
```

G4.3 Explain cache replacement algorithms. Differentiate between SRAM and DRAM. Describe the cache performance calculation.	**2080 Ashwin (Back) | 2079 Chaitra (Regular) | 2076 Baisakh (Regular) [4+4]**

ANSWER

Cache Replacement Algorithms:

When a cache miss occurs and the cache is full, a replacement algorithm selects which existing cache block to evict to make room for the new block.

1. LRU (Least Recently Used): Evicts the block not accessed for the longest time. Requires tracking access time/order for all blocks. Best hit rate but complex hardware for large caches (needs LRU counter per set).

2. FIFO (First-In First-Out): Evicts the block that has been in cache the longest (regardless of usage). Simple implementation (circular pointer). Can suffer Belady's anomaly (more frames → more misses).

3. LFU (Least Frequently Used): Evicts block with fewest accesses. Keeps frequently accessed blocks. Risk: old blocks with high early access count never evicted (aging problem).

4. Random: Randomly selects block to evict. Very simple hardware. Surprisingly good average performance (no pathological cases).

5. Optimal (Belady's): Evicts block that will not be used for the longest future time. Optimal hit rate but requires future knowledge — impossible in practice, used as benchmark.

Property	SRAM	DRAM
Storage element	6-transistor flip-flop per bit	1 transistor + 1 capacitor per bit
Refresh required	No — holds data as long as powered	Yes — capacitor leaks, must refresh every ~64ms
Speed (access time)	Very fast: 0.5–10 ns	Slow: 50–100 ns
Density	Low (6T per bit)	Very high (1T+1C per bit)
Power	Higher static power (flip-flops)	Lower static; refresh power overhead
Cost	Expensive per bit	Cheap per bit
Used for	CPU cache (L1, L2, L3)	Main system RAM

Cache Performance Calculation:

```
Average Memory Access Time (AMAT):
  AMAT = Hit Time + Miss Rate × Miss Penalty

Example: Cache A vs Cache B comparison
  Cache A: size=4KB, miss rate=15%, hit cost=3 cycles, miss cost=25 cycles
  Cache B: size=12KB, miss rate=5%, hit cost=6 cycles, miss cost=25 cycles

  AMAT_A = 3 + 0.15 × 25 = 3 + 3.75 = 6.75 cycles
  AMAT_B = 6 + 0.05 × 25 = 6 + 1.25 = 7.25 cycles

  Cache A is better (lower AMAT) despite smaller size!
  The faster hit time more than compensates for higher miss rate.
```

Chapter 5 Gap Topics: Interfacing

G5.1 Explain port-based I/O and bus-based I/O. Compare strobe and handshake protocols for data transfer.	2076 Baisakh (Regular) | 2072 Magh (Back) | 2071 Bhadra (Regular) [3+5]

ANSWER

Port-Based I/O:

The processor communicates with peripherals through dedicated I/O ports (registers) mapped to specific addresses. The processor uses special I/O instructions (IN, OUT on x86) or memory-mapped I/O addresses to read/write peripheral registers.

- Example: 8051 — P0, P1, P2, P3 are 8-bit I/O port registers at SFR addresses 80H, 90H, A0H, B0H.
- Simple, low cost, works well for single-board microcontrollers.
- Limited scalability — adding more peripherals requires more port addresses.

Bus-Based I/O:

Peripherals connect to a shared system bus (address bus + data bus + control bus). Each peripheral has a unique address range. Multiple peripherals share the same bus with bus arbitration.

- Example: PCI bus, USB hub, I2C bus, SPI bus — all support multiple devices on shared interconnect.
- Scalable — add peripherals without processor redesign. Supports DMA for high-speed transfers.
- More complex (bus arbitration, protocol overhead).

Property	Port-Based I/O	Bus-Based I/O
Connection	Dedicated port pins	Shared bus wires
Scalability	Limited (fixed ports)	High (add devices on bus)
Speed	Fast (direct access)	Varies (bus sharing overhead)
Complexity	Simple	More complex (arbitration, protocol)
Example	8051 Ports P0-P3	PCI, USB, I2C, SPI, CAN

Strobe Protocol:

```
Unidirectional timing protocol — sender controls timing completely:

Sender:    Data ——————————————————[VALID DATA]——————————————————
```

```
Sender:    STB  ──────────────────┐            ┌────────────────
                                  └────────────┘
                              STB LOW = data valid window

Receiver samples data on STB low (or rising/falling edge of STB)

Problem: If receiver is busy, it may miss the strobe → no feedback
         Sender doesn't know if data was received successfully
```

Handshake Protocol:

```
Bidirectional protocol — receiver acknowledges receipt:

Sender:    Data ──────────────────[VALID DATA]──────────────────
Sender:    STB  ────┐                          ┌────────────────
                    └──────────────────────────┘
Receiver:  ACK  ────────────────────┐
                                    └────────────────────────


Sequence:
  1. Sender puts data on bus, asserts STB (data valid)
  2. Receiver reads data, asserts ACK (data received)
  3. Sender deasserts STB (transfer complete)
  4. Receiver deasserts ACK (ready for next transfer)

Advantage: Reliable — sender waits for ACK before next transfer
           Works for any receiver speed (self-synchronizing)
Disadvantage: Slower than strobe (requires round-trip ACK)
```

Chapter 6 Gap Topics: RTOS

G6.1 Explain mutual exclusion in RTOS. Describe the sleep-and-wake mechanism for task synchronization. Differentiate between user-level and kernel-level threads.	**2075 Bhadra (Back) | 2072 Magh (Back) | 2071 Bhadra (Regular) [4+4]**

ANSWER

Mutual Exclusion:

Mutual exclusion (mutex) ensures that only one task/thread can access a shared resource (critical section) at a time, preventing race conditions and data corruption.

- Critical section: A code region that accesses shared data/hardware and must not be interrupted or accessed concurrently by other tasks.
- Mutex: A binary lock — only the task that acquires (locks) the mutex can enter the critical section. Other tasks trying to lock it are blocked until it is released.

```
// C pseudocode for mutual exclusion
mutex_t lock;
mutex_init(&lock);

Task A:                          Task B:
  mutex_lock(&lock);               mutex_lock(&lock);  // blocks if A holds
  // critical section              // critical section
  shared_resource++;               shared_resource--;
  mutex_unlock(&lock);             mutex_unlock(&lock);
```

Sleep-and-Wake Synchronization:

A producer-consumer synchronization mechanism where a task sleeps (blocks itself) when it has nothing to do, and is woken by another task when work is available.

```
Consumer task:                   Producer task:
  while(1) {                       while(1) {
    mutex_lock(&buf_lock);            // produce item
    if (buffer_empty()) {             mutex_lock(&buf_lock);
      sleep();   // block self          put_item(buffer, item);
    }                                 mutex_unlock(&buf_lock);
    item = get_item(buffer);          wake(consumer); // wake consumer
    mutex_unlock(&buf_lock);       }
    process(item);
  }

Advantage: No CPU wasted on busy-waiting (polling)
Disadvantage: Sleep/wake can have race condition if not atomically
implemented
```

Property	User-Level Threads	Kernel-Level Threads
Implementation	Thread library in user space (no OS kernel involvement)	OS kernel manages and schedules threads directly
Context switch	Very fast (no system call — just library call)	Slower (requires mode switch to kernel)
Blocking	If one thread blocks (e.g., I/O), entire process blocks	Other threads can continue running (kernel schedules them)
Parallelism	No true parallelism on SMP (kernel sees only one process)	True parallel execution on multi-core systems
Examples	POSIX pthreads (user-level mode), Java green threads	Linux NPTL threads, Windows threads, RTOS tasks

G6.2 What is multithreading? Explain its advantages. Write a simple multithreading program in C using pthreads. Consider six processes P1–P6 and compute scheduling (preemptive priority + SJF).	**2080 Ashwin (Back) | 2072 Magh (Back) | 2079 Ashwin (Back) [3+3+6]**

ANSWER

Multithreading:

Multithreading is the ability of a CPU or RTOS to execute multiple threads of execution concurrently within a single process, sharing the same memory space and resources but with independent execution flows (PC, stack, registers).

Advantages:

- Improved responsiveness: GUI thread continues responding to user while worker thread does computation.
- Resource sharing: Threads share process address space — no IPC overhead for shared data (vs. multiprocessing).
- Economy: Creating a thread is faster and cheaper than creating a new process.
- Utilization of multi-core: True parallel execution on SMP systems — multiple threads run on different cores simultaneously.
- Simplified program structure: Complex concurrent tasks modeled naturally as separate threads.

```
// Simple pthreads multithreading program in C
#include <stdio.h>
#include <pthread.h>

void *thread_func(void *arg) {
    int id = *(int*)arg;
    printf("Thread %d: running\n", id);
    return NULL;
}
```

```
int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;
    pthread_create(&t1, NULL, thread_func, &id1); // create thread 1
    pthread_create(&t2, NULL, thread_func, &id2); // create thread 2
    pthread_join(t1, NULL);  // wait for thread 1 to finish
    pthread_join(t2, NULL);  // wait for thread 2 to finish
    printf("Both threads done\n");
    return 0;
}
```

6-Process Preemptive Priority Scheduling Numerical (2079 Ashwin):

Processes P1–P4 enter at t=0; P5 at t=1ms; P6 at t=3ms of P4's start. Priority 0=highest.

Process	Arrival	Burst	Priority
P1	0	5	0
P2	0	4	2
P3	0	6	4
P4	0	4	5
P5	1	2	1
P6	3 (of P4)	3	3

```
Execution timeline (preemptive priority — 0=highest):
t=0:  P1(pr=0) starts (highest priority of {P1,P2,P3,P4})
t=1:  P5(pr=1) arrives → P1(pr=0) still highest → P1 continues
t=5:  P1 finishes. Ready: P2(pr=2),P3(pr=4),P4(pr=5),P5(pr=1)
      P5(pr=1) runs next
t=7:  P5 finishes. P4 has been running... wait:
      Actually at t=3 of P4 execution (absolute t=3), P6 arrives.
      Simplified: after P1,P5 complete, schedule by priority:
      P2(pr=2) → P6(pr=3) → P3(pr=4) → P4(pr=5)

Gantt: |P1:0-5|P5:5-7|P2:7-11|P6:11-14|P3:14-20|P4:20-24|

Waiting Times:
  P1: 0ms (ran first)
  P2: 7ms (arrived t=0, ran t=7)
  P3: 14ms (arrived t=0, ran t=14)
  P4: 20ms (arrived t=0, ran t=20)
  P5: 5-1 = 4ms (arrived t=1, ran t=5)
  P6: 11-3 = 8ms (arrived t=3, ran t=11)
AWT = (0+7+14+20+4+8)/6 = 53/6 ≈ 8.83 ms
```

Chapter 7 Gap Topics: Control System

G7.1 Discuss the practical issues with computer-based control systems. List the benefits of computer control implementation. Explain PD and PI controller design.	**2081 Ashwin (Back) | 2074 Bhadra (Regular) | 2071 Bhadra (Regular) [6+2 / 3+5]**

ANSWER

Practical Issues with Computer-Based Control:

- Sampling and quantization: Continuous physical signals must be sampled (ADC). Sampling rate must satisfy Nyquist theorem (fs ≥ 2×fbandwidth). Quantization error introduces noise.
- Computational delay: The PID computation, ADC conversion, and DAC output introduce a time delay (latency) in the control loop, which can destabilize the system if too large.
- Fixed-point arithmetic: Embedded systems often use fixed-point (no FPU) — PID integral and derivative terms require careful scaling to avoid overflow and precision loss.
- Anti-windup: Integral term accumulates when output is saturated (actuator at limit). Without anti-windup, the controller takes a long time to recover when saturation ends.
- Sensor noise: Derivative term amplifies high-frequency noise from sensors. Requires low-pass filtering or derivative-on-measurement to avoid actuator chatter.
- Actuator saturation and nonlinearity: Real actuators have limits (0–100% valve, ±12V motor). Controller must handle saturation gracefully.
- Power supply stability: Embedded control systems may face voltage fluctuations — ADC reference voltage variations directly affect measurement accuracy.

Benefits of Computer-Based Control:

- Flexibility: Algorithm changes via software update without hardware redesign — field upgrades possible.
- Precision: Higher precision than analog controllers; repeatable, drift-free behavior.
- Multiple-loop control: One processor can implement dozens of PID loops simultaneously.
- Data logging and monitoring: Real-time data recording for fault diagnosis and performance analysis.
- Communication: Digital controllers integrate easily with industrial networks (CAN, Modbus, Ethernet).

PI Controller (Proportional-Integral):

```
Transfer function: C(s) = Kp + Ki/s = (Kp·s + Ki) / s
```

```
Use: Eliminates steady-state error (integral term).
      Suitable when noise makes derivative impractical.

Time domain: u(t) = Kp·e(t) + Ki·∫e(t)dt

Effect vs P-only:
  + Steady-state error eliminated (integral forces error→0)
  + Handles constant load disturbances
  - Slower response (phase lag from integrator)
  - Risk of integral windup if output saturates
```

PD Controller (Proportional-Derivative):

```
Transfer function: C(s) = Kp + Kd·s

Use: Fast systems needing damping; when no steady-state error required.

Time domain: u(t) = Kp·e(t) + Kd·(de/dt)

Effect vs P-only:
  + Reduces overshoot and oscillation (derivative predicts error)
  + Faster settling time
  - Does NOT eliminate steady-state error (no integrator)
  - Very sensitive to sensor noise (derivative amplifies noise)
  - In practice: add low-pass filter: Kd·s/(τf·s + 1)
```

Chapter 8 Gap Topics: IC Technology

G8.1 Draw the top-down view and schematic for the function F = xz + yz'. Describe the steps of photolithography using positive photoresist.	**2075 Bhadra (Back) | 2079 Chaitra (Regular) [5+3]**

ANSWER

Logic Implementation of F = xz + yz':

```
Boolean expression: F = xz + yz'

Gate-level schematic:
  x ────┐
        ├── AND1  ──────────┐
  z ────┘                   ├── OR ──▶ F
                            │
  Y  ────┐                  │
         ├── AND2 ──────────┘
 z──[NOT]┘
         z' (z complement)

Truth table verification:
  x y z | z' | xz | yz' | F=xz+yz'
  0 0 0 |  1 |  0 |  0  |  0
  0 0 1 |  0 |  0 |  0  |  0
  0 1 0 |  1 |  0 |  1  |  1
  0 1 1 |  0 |  0 |  0  |  0
  1 0 0 |  1 |  0 |  0  |  0
  1 0 1 |  0 |  1 |  0  |  1
  1 1 0 |  1 |  0 |  1  |  1
  1 1 1 |  0 |  1 |  0  |  1
```

CMOS Layout (Top-Down View) Layers:

```
Layer stack (bottom to top):
  1. P-substrate (background)
  2. N-well region (for PMOS transistors)
  3. Active/Diffusion layer (where transistors are formed)
  4. Polysilicon layer (gate electrode — crosses active = transistor)
  5. Contact cuts (connects poly/diffusion to Metal1)
  6. Metal1 (first interconnect layer — power rails + local wiring)
  7. Via1 (connects Metal1 to Metal2)
  8. Metal2 (second interconnect layer)

Key design rules for F = xz + yz':
  • PMOS transistors in N-well (pull-up network)
```

```
  • NMOS transistors in P-substrate (pull-down network)
  • VDD rail at top, GND rail at bottom
  • Inputs x, y, z as polysilicon gates crossing N-well/P-sub boundaries
```

Positive Photoresist in Photolithography:

- Positive resist: UV-exposed areas become SOLUBLE in developer (exposed = removed).
- The pattern on the mask (chrome = opaque, clear = transparent) is transferred as: clear areas on mask → exposed resist → dissolved away → underlying material etched in those areas.
- The final pattern on wafer MATCHES the clear (transparent) areas of the mask.

```
Step 1: Apply positive photoresist (spin coat, ~1µm)
Step 2: Soft bake (90°C, evaporate solvent)
Step 3: Align mask; expose with UV through mask
         Clear mask area → PR exposed (becomes soluble)
         Chrome mask area → PR unexposed (stays insoluble)
Step 4: Develop: exposed PR dissolves → pattern revealed
Step 5: Hard bake (120°C)
Step 6: Etch: exposed underlying material removed; PR protects rest
Step 7: Strip resist

Result: Etched pattern matches CLEAR areas of original mask
```

Chapter 9 Gap Topics: 8051 Microcontroller

G9.1 Write an 8051 assembly program to display digits 9 to 0 (count down) on a Common Anode 7-segment display connected at Port 1. Explain Common Anode configuration.	2080 Chaitra (Back) | 2073 Bhadra (Back) [2+6]

ANSWER

Common Anode (CA) 7-Segment Display:

- In Common Anode configuration, all segment anodes are connected together to VCC (+5V).
- A segment lights up when its CATHODE is driven LOW (logic 0). So segment codes are ACTIVE LOW (inverted from Common Cathode).
- To turn a segment ON: output 0 to that bit. To turn it OFF: output 1.

Common Anode Segment Codes (active LOW — complement of CC codes):

Digit	CC Code (active HIGH)	CA Code (active LOW = NOT of CC)
0	3FH = 0011 1111	C0H = 1100 0000
1	06H = 0000 0110	F9H = 1111 1001
2	5BH = 0101 1011	A4H = 1010 0100
3	4FH = 0100 1111	B0H = 1011 0000
4	66H = 0110 0110	99H = 1001 1001
5	6DH = 0110 1101	92H = 1001 0010
6	7DH = 0111 1101	82H = 1000 0010
7	07H = 0000 0111	F8H = 1111 1000
8	7FH = 0111 1111	80H = 1000 0000
9	6FH = 0110 1111	90H = 1001 0000

```
;————————————————————————————————————————————————————————————
; Display 9→0 countdown on Common Anode 7-seg at Port 1
; Port 1 drives segment cathodes (active LOW for CA)
; ~500ms delay between each digit
;————————————————————————————————————————————————————————————
        ORG   0000H

SEG_CA: DB  C0H,F9H,A4H,B0H,99H,92H,82H,F8H,80H,90H
        ;    0   1   2   3   4   5   6   7   8   9  (CA codes)

        ORG   0100H
```

```
START:  MOV   DPTR, #SEG_CA    ; point to CA segment table
        MOV   R0,   #09H       ; start from digit 9

LOOP:   MOV   A, R0            ; A = current digit (9 down to 0)
        MOVC  A, @A+DPTR       ; lookup CA segment code
        MOV   P1, A            ; output to Port 1 (active LOW)
        ACALL DELAY            ; ~500ms delay
        DJNZ  R0, LOOP         ; decrement and loop until R0=0
        ; When R0=0 (digit 0), DJNZ won't jump (reaches 0 and stops)
        ; Display 0 for one more interval then restart:
        SJMP  START            ; restart from 9

DELAY:  MOV   R1, #10
D1:     MOV   R2, #200
D2:     MOV   R3, #250
D3:     DJNZ  R3, D3
        DJNZ  R2, D2
        DJNZ  R1, D1
        RET
        END
```

G9.2 Write an 8051 assembly program to read data from switches connected at Port 1 and send it to Port 2 for LED display. Also write a program to generate a 75% duty cycle pulse at P1.7 when switch at P1.1 is ON.

2073 Magh (Back) | 2079 Jestha (Back) | 2074 Bhadra (Regular) [4+4]

ANSWER

(a) Read switches at Port 1, display on Port 2 LEDs:

```
;————————————————————————————————————————————————————————————
; Read 8 switches connected at Port 1
; Display switch states on 8 LEDs at Port 2
; Port 1: configure as input (write 0FFH first to make it input)
; Port 2: output (LED ON when bit=0 for active-LOW, or bit=1 for active-
HIGH)
; Assuming active-HIGH LEDs: LED ON when P2 bit = 1
;————————————————————————————————————————————————————————————
        ORG   0000H

START:  MOV   P1,  #0FFH   ; Configure Port 1 as input (write all 1s)
        MOV   A,   P1      ; Read switch states from Port 1
        MOV   P2,  A       ; Send to Port 2 (LED display)
        SJMP  START        ; Repeat continuously (polling)
        END

; Note: In 8051, all ports are quasi-bidirectional.
; Writing 0FFH to P1 enables input mode (internal pull-ups ON).
```

```
; Switch open → pin reads 1; Switch closed to GND → pin reads 0.
```

(b) 75% Duty Cycle Pulse at P1.7 (when switch P1.1 is ON):

```
;──────────────────────────────────────────────────────────────────
; 75% duty cycle square wave at P1.7
; Controlled by switch at P1.1 (P1.1=0 means switch pressed/ON)
; 75% duty = HIGH for 3 units, LOW for 1 unit (period = 4 units)
; Using 500µs unit delay → period = 2ms, frequency = 500 Hz
;──────────────────────────────────────────────────────────────────
        ORG   0000H

START:  MOV   P1, #0FFH    ; P1 as input (to read switch P1.1)

CHECK:  JB    P1.1, CHECK  ; Wait until P1.1=0 (switch pressed)
                           ; JB = Jump if Bit set (1=switch open)
                           ; Loop here while switch is OFF

; Switch is ON (P1.1=0) — generate 75% duty cycle
WAVE:   SETB  P1.7         ; P1.7 HIGH (start of cycle)
        ACALL DELAY_500US  ; HIGH for 500µs  (1st unit)
        ACALL DELAY_500US  ; HIGH for 500µs  (2nd unit)
        ACALL DELAY_500US  ; HIGH for 500µs  (3rd unit) → total 1.5ms HIGH
        CLR   P1.7         ; P1.7 LOW
        ACALL DELAY_500US  ; LOW for 500µs   (4th unit) → 0.5ms LOW
        JNB   P1.1, WAVE   ; If switch still ON, repeat
        SJMP  CHECK        ; Switch released, go back to check

; 500 microsecond delay at 11.0592 MHz
DELAY_500US:
        MOV   R0, #230     ; ~500µs: 230 × 2 cycles × 1.085µs ≈ 499µs
DLOOP:  DJNZ  R0, DLOOP
        RET
        END

; Duty cycle verification:
; HIGH time = 3 × 500µs = 1500µs
; LOW time  = 1 × 500µs =  500µs
; Period = 2000µs = 2ms  → f = 500 Hz
; Duty cycle = 1500/2000 = 75% ✓
```

G9.3 Write an 8051 assembly program for a down counter displaying 99 to 00 on two 7-segment displays. Also write a program for a 5ms precision delay.

2077 Baisakh (Back) | 2071 Magh (Back) [4+4]

ANSWER

(a) Down Counter 99 to 00 on Two 7-Segment Displays:

```
;————————————————————————————————————————————————————————————
; Down counter: 99 → 98 → ... → 01 → 00 → 99 (repeats)
; Port 0 = tens digit (MSB), Port 2 = units digit (LSB)
; Common Cathode display (active HIGH)
;————————————————————————————————————————————————————————————
        ORG   0000H
        LJMP  START

SEG_CC: DB  3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH
        ;    0   1   2   3   4   5   6   7   8   9

        ORG   0100H
START:  MOV   DPTR, #SEG_CC
        MOV   R2,   #09H    ; tens digit starts at 9

TENS:   MOV   R3,   #09H    ; units digit starts at 9

UNITS:  ; Display tens digit on Port 0
        MOV   A,  R2
        MOVC  A,  @A+DPTR
        MOV   P0, A

        ; Display units digit on Port 2
        MOV   A,  R3
        MOVC  A,  @A+DPTR
        MOV   P2, A

        ACALL DELAY_1S     ; 1 second between counts

        DJNZ  R3, UNITS    ; decrement units; if not 0, loop
        DJNZ  R2, TENS     ; decrement tens; if not 0, loop
        SJMP  START        ; restart from 99

; ~1 second delay at 11.0592 MHz
DELAY_1S: MOV R4,#50
D1:       MOV R5,#200
D2:       MOV R6,#250
D3:       DJNZ R6,D3
          DJNZ R5,D2
          DJNZ R4,D1
          RET
        END
```

(b) Precise 5ms Delay using Timer 0 Mode 1 (12 MHz crystal):

```
;————————————————————————————————————————————————————————————
```

```
; Precise 5ms delay using Timer 0 Mode 1 @ 12 MHz crystal
; Machine cycle = 1µs; counts needed = 5000
; Timer initial value = 65536 - 5000 = 60536 = EC78H
; TH0 = ECH, TL0 = 78H
;————————————————————————————————————————————————————————————
        ORG   0000H

MAIN:   ACALL DELAY_5MS    ; call 5ms delay
        CPL   P1.0         ; toggle pin P1.0 every 5ms → 100Hz square wave
        SJMP  MAIN

DELAY_5MS:
        MOV   TMOD, #01H   ; Timer 0, Mode 1 (16-bit)
        MOV   TH0,  #0ECH  ; 60536 = EC78H → counts for 5ms
        MOV   TL0,  #78H
        SETB  TR0           ; start Timer 0
WAIT:   JNB   TF0, WAIT   ; wait for Timer 0 overflow (TF0=1)
        CLR   TF0           ; clear overflow flag
        CLR   TR0           ; stop Timer 0
        RET
        END

; Verification: 65536 - 60536 = 5000 counts × 1µs = 5ms ✓
; For 11.0592 MHz: MC period = 1.085µs
;   5ms / 1.085µs = 4608 counts
;   65536 - 4608 = 60928 = EE00H → TH0=EEH, TL0=00H
```

Chapter 10 Gap Topics: VHDL

G10.1 What is a PROCESS in VHDL? How does behavioral modeling differ from dataflow modeling? Write VHDL code for a 1-to-8 DeMUX.	2080 Ashwin (Back) | 2080 Chaitra (Back) [2+2+4]

ANSWER

PROCESS in VHDL:

A PROCESS is a concurrent statement in VHDL that contains a sequence of sequential statements (like software code). It executes whenever any signal in its sensitivity list changes.

- Sensitivity list: The list of signals in parentheses after PROCESS keyword. The process re-executes when any listed signal changes value.
- Sequential inside: All statements inside PROCESS execute in order (top to bottom) — like a software function.
- Concurrent outside: Multiple PROCESS statements in an architecture all execute concurrently (in parallel) — they model parallel hardware.

```
process(A, B, sel)          -- re-run when A, B, or sel changes
begin
  if sel = '0' then
    Y <= A;                 -- sequential assignment
  else
    Y <= B;
  end if;
end process;
```

Aspect	Behavioral Model	Dataflow Model
Constructs	PROCESS, if/else, case, loop	Concurrent signal assignments (<=), Boolean ops
Execution	Sequential inside PROCESS	All assignments execute concurrently
Abstraction	High — describes algorithm	Medium — describes logic expressions
Best for	FSMs, sequential logic, complex control	Combinational logic, simple gates
Example	if A='1' then Y<='0'; end if;	Y <= A AND B OR (NOT C);

```
-- ═══════════════════════════════════════════════════
-- 1-to-8 DeMUX (Demultiplexer)
-- One input Y routed to one of 8 outputs based on 3-bit sel
-- All other outputs = '0'
-- ═══════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity DeMUX1to8 is
  Port(
    Y   : in  STD_LOGIC;                     -- single input
    sel : in  STD_LOGIC_VECTOR(2 downto 0); -- 3-bit select
    O   : out STD_LOGIC_VECTOR(7 downto 0)  -- 8 outputs
  );
end DeMUX1to8;

architecture Behavioral of DeMUX1to8 is
begin
  process(Y, sel)
  begin
    O <= (others => '0');    -- default: all outputs = 0
    case sel is
      when "000" => O(0) <= Y;
      when "001" => O(1) <= Y;
      when "010" => O(2) <= Y;
      when "011" => O(3) <= Y;
      when "100" => O(4) <= Y;
      when "101" => O(5) <= Y;
      when "110" => O(6) <= Y;
      when "111" => O(7) <= Y;
      when others => O <= (others => '0');
    end case;
  end process;
end Behavioral;
```

G10.2 Write a VHDL code for a Decoder (3-to-8 decoder). Also write VHDL code for a BCD counter displaying 0 to 9999 on 7-segment displays.

2076 Baisakh (Regular) | 2070 Bhadra (Back) [2+6]

ANSWER

```
-- ═══════════════════════════════════════════════════════
-- 3-to-8 DECODER — Behavioral VHDL
-- Input: 3-bit binary; Output: one of 8 lines HIGH (active HIGH)
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Decoder3to8 is
  Port(
    A   : in  STD_LOGIC_VECTOR(2 downto 0);  -- 3-bit input
    en  : in  STD_LOGIC;                       -- enable (active HIGH)
    Y   : out STD_LOGIC_VECTOR(7 downto 0)    -- 8 outputs
  );
end Decoder3to8;
```

```
architecture Behavioral of Decoder3to8 is
begin
  process(A, en)
  begin
    if en = '1' then
      case A is
        when "000" => Y <= "00000001";
        when "001" => Y <= "00000010";
        when "010" => Y <= "00000100";
        when "011" => Y <= "00001000";
        when "100" => Y <= "00010000";
        when "101" => Y <= "00100000";
        when "110" => Y <= "01000000";
        when "111" => Y <= "10000000";
        when others => Y <= "00000000";
      end case;
    else
      Y <= (others => '0');   -- disabled: all outputs LOW
    end if;
  end process;
end Behavioral;
```

```
-- ═══════════════════════════════════════════════════════
-- BCD COUNTER 0 to 9999 — Behavioral VHDL
-- 4 BCD digits: thousands, hundreds, tens, units
-- Each digit counts 0-9, then carries to next digit
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity BCD_Counter_9999 is
  Port(
    clk      : in  STD_LOGIC;
    reset    : in  STD_LOGIC;
    units    : out STD_LOGIC_VECTOR(3 downto 0);  -- 0-9
    tens     : out STD_LOGIC_VECTOR(3 downto 0);  -- 0-9
    hundreds : out STD_LOGIC_VECTOR(3 downto 0);  -- 0-9
    thousands: out STD_LOGIC_VECTOR(3 downto 0)   -- 0-9
  );
end BCD_Counter_9999;

architecture Behavioral of BCD_Counter_9999 is
  signal u, t, h, th : UNSIGNED(3 downto 0) := (others => '0');
begin
  process(clk, reset)
  begin
```

```
    if reset = '1' then
      u <= "0000"; t <= "0000"; h <= "0000"; th <= "0000";
    elsif rising_edge(clk) then
      -- Units digit
      if u = 9 then
        u <= "0000";
        -- Tens digit carry
        if t = 9 then
          t <= "0000";
          -- Hundreds digit carry
          if h = 9 then
            h <= "0000";
            -- Thousands digit carry
            if th = 9 then
              th <= "0000";   -- rollover 9999 → 0000
            else
              th <= th + 1;
            end if;
          else
            h <= h + 1;
          end if;
        else
          t <= t + 1;
        end if;
      else
        u <= u + 1;
      end if;
    end if;
  end process;

  units     <= STD_LOGIC_VECTOR(u);
  tens      <= STD_LOGIC_VECTOR(t);
  hundreds  <= STD_LOGIC_VECTOR(h);
  thousands <= STD_LOGIC_VECTOR(th);
end Behavioral;
```

G10.3 Design a sequence detector for the string '1101' using an FSM. Show the state diagram and implement in VHDL. Also design an elevator controller FSM.	**2070 Magh (Back) | 2079 Chaitra (Regular) [3+5]**

ANSWER

Sequence Detector for '1101' — Moore FSM:

```
State diagram (Moore — output depends only on state):

States: S0(init) → S1(got 1) → S2(got 11) → S3(got 110) → S4(got
1101=DETECT)
```

```
Transitions:
  S0: input=1 → S1;  input=0 → S0
  S1: input=1 → S2;  input=0 → S0
  S2: input=0 → S3;  input=1 → S2 (stay — got '11', last '1' counts)
  S3: input=1 → S4;  input=0 → S0
  S4: input=1 → S2;  input=0 → S0  (output=1 in S4, then move on)
```

```
-- ═══════════════════════════════════════════════════════
-- SEQUENCE DETECTOR for '1101' — Moore FSM
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SeqDet_1101 is
  Port(
    clk      : in  STD_LOGIC;
    reset    : in  STD_LOGIC;
    x        : in  STD_LOGIC;   -- serial input
    detected : out STD_LOGIC    -- '1' when 1101 detected
  );
end SeqDet_1101;

architecture Behavioral of SeqDet_1101 is
  type state_t is (S0, S1, S2, S3, S4);
  signal state : state_t := S0;
begin
  process(clk, reset)
  begin
    if reset = '1' then  state <= S0;
    elsif rising_edge(clk) then
      case state is
        when S0 => if x='1' then state<=S1; else state<=S0; end if;
        when S1 => if x='1' then state<=S2; else state<=S0; end if;
        when S2 => if x='0' then state<=S3; else state<=S2; end if;
        when S3 => if x='1' then state<=S4; else state<=S0; end if;
        when S4 => if x='1' then state<=S2; else state<=S0; end if;
      end case;
    end if;
  end process;
  detected <= '1' when state = S4 else '0';  -- Moore output
end Behavioral;
```

Elevator Controller FSM (2-floor simplified):

```
-- ═══════════════════════════════════════════════════════
-- ELEVATOR CONTROLLER — 3-floor FSM
-- States: FLOOR1, MOVING_UP, FLOOR2, MOVING_UP2, FLOOR3, MOVING_DOWN...
```

```
-- Simplified: responds to floor_req input
-- ==========================================================
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Elevator_FSM is
  Port(
    clk       : in  STD_LOGIC;
    reset     : in  STD_LOGIC;
    req       : in  STD_LOGIC_VECTOR(2 downto 0); -- floor requests b2b1b0
    door_open : out STD_LOGIC;  -- 1=door open
    motor_up  : out STD_LOGIC;  -- 1=moving up
    motor_dn  : out STD_LOGIC   -- 1=moving down
  );
end Elevator_FSM;

architecture Behavioral of Elevator_FSM is
  type state_t is (AT_F1, GOING_UP, AT_F2, GOING_UP2, AT_F3, GOING_DN,
GOING_DN2);
  signal state : state_t := AT_F1;
begin
  process(clk, reset)
  begin
    if reset='1' then state <= AT_F1;
    elsif rising_edge(clk) then
      case state is
        when AT_F1 =>
          if req(1)='1' or req(2)='1' then state <= GOING_UP; end if;
        when GOING_UP =>  state <= AT_F2;
        when AT_F2 =>
          if req(2)='1' then state <= GOING_UP2;
          elsif req(0)='1' then state <= GOING_DN;
          end if;
        when GOING_UP2 => state <= AT_F3;
        when AT_F3 =>
          if req(0)='1' or req(1)='1' then state <= GOING_DN; end if;
        when GOING_DN  => state <= AT_F2;
        when GOING_DN2 => state <= AT_F1;
      end case;
    end if;
  end process;
  door_open <= '1' when (state=AT_F1 or state=AT_F2 or state=AT_F3) else
'0';
  motor_up  <= '1' when (state=GOING_UP or state=GOING_UP2) else '0';
  motor_dn  <= '1' when (state=GOING_DN or state=GOING_DN2) else '0';
end Behavioral;
```

Chapter 1 — Additional Gap Topics

G1.4 Design a custom single-purpose processor to compute the mean (average) of n numbers entered by the user. Show the algorithm, FSMD state diagram, datapath, and controller block diagram.	**2081 Ashwin (Back) | 2079 Ashwin (Back) [8]**

ANSWER

Algorithm:

1. Initialize: sum=0, count=0.
2. While more inputs: read next number x; sum = sum + x; count = count + 1.
3. Mean = sum / count. Output mean.

FSMD State Diagram:

```
IDLE —(go=1)→ INIT ———→ READ —(more=1)→ ADD → INC → READ
              (sum=0,     |                                |
               cnt=0)     | more=0                         |
                          ▼                                |
                        DIVIDE ———————————————————————————
                       (mean=sum/cnt)
                          |
                          ▼
                        DONE (output mean, done=1)
```

Datapath:

```
Registers:
  Rsum  (16-bit) — accumulates sum of inputs
  Rcnt  (8-bit)  — counts number of inputs
  Rdata (8-bit)  — holds current input
  Rmean (8-bit)  — stores result mean

Functional units:
  Adder: Rsum ← Rsum + Rdata         (in ADD state)
  Incrementer: Rcnt ← Rcnt + 1       (in INC state)
  Divider: Rmean ← Rsum / Rcnt       (in DIVIDE state)
  Comparator: zero_flag ← (more=0)   (from input 'more' signal)

Controller outputs: ld_sum, ld_cnt, ld_data, ld_mean, done
Controller inputs:  go, more (indicates another input available)
```

G1.5 Design a single-purpose processor to determine the sum of digits of an integer. Show the algorithm, FSMD, datapath, and controller.	**2076 Baisakh (Regular) [8]**

ANSWER

Algorithm:

1. Load integer N. Initialize sum=0.
2. While N > 0: digit = N mod 10; sum = sum + digit; N = N / 10.
3. Output sum.

FSMD State Diagram:

```
IDLE —(go=1)→ INIT ———→ CHECK —(N=0)→ DONE
              (sum=0,      |            (out=sum, done=1)
               N=input)    | N>0
                           ▼
                          EXTRACT —→ ACCUMULATE —→ SHIFT —→ CHECK
                         (dig=N      (sum=sum+dig)  (N=N/10)
                          mod 10)
```

Datapath:

```
Registers: RN (input number), Rsum, Rdig
Units:
  Modulo-10:   Rdig ← RN mod 10   (divide-by-10 remainder)
  Adder:       Rsum ← Rsum + Rdig
  Div-by-10:   RN   ← RN / 10     (integer division)
  Comparator:  zero_flag ← (RN = 0)

Example trace: N = 235
  iter1: dig=235 mod 10=5, sum=0+5=5, N=235/10=23
  iter2: dig=23  mod 10=3, sum=5+3=8, N=23/10=2
  iter3: dig=2   mod 10=2, sum=8+2=10, N=2/10=0 → DONE
  Output: sum = 10 ✓ (2+3+5=10)
```

G1.6 Develop an algorithm, draw the state diagram, and design the datapath of a custom single-purpose processor that determines the largest of four integers.	**2073 Bhadra (Back) [8]**

ANSWER

Algorithm:

1. Load four integers a, b, c, d into registers Ra, Rb, Rc, Rd.
2. max = Ra.
3. If Rb > max: max = Rb.
4. If Rc > max: max = Rc.
5. If Rd > max: max = Rd.
6. Output max.

FSMD State Diagram:

```
IDLE —(go=1)→ LOAD → COMP_B → COMP_C → COMP_D → DONE
              (Ra=a,    (if b>max:  (if c>max:  (if d>max:  (out=max)
               Rb=b,     max←Rb)    max←Rc)     max←Rd)
               Rc=c,
               Rd=d,
               max←Ra)
```

Datapath:

```
Registers: Ra, Rb, Rc, Rd (8-bit each), Rmax (8-bit)
Units:
  Comparator: gt_flag ← (Rx > Rmax)
  MUX: Rmax ← Ra (LOAD) or Rb (COMP_B, if gt) or Rc (COMP_C, if gt)
               or Rd (COMP_D, if gt) or Rmax (if not gt)

Example trace: a=5, b=9, c=3, d=7
  LOAD:   max=5
  COMP_B: 9>5 → max=9
  COMP_C: 3>9? No → max=9
  COMP_D: 7>9? No → max=9
  Output: 9 ✓
```

G1.7 Develop an algorithm, draw the FSMD, and design the datapath of a custom single-purpose processor that outputs 1 if the input integer is prime, or 0 if it is not.	**2078 Poush (Back) [8]**

ANSWER

Algorithm (Trial Division):

1. Input N. If N ≤ 1 → not prime (output 0).
2. Initialize divisor d = 2.
3. While d × d ≤ N: if N mod d = 0 → not prime (output 0). Else d = d + 1.
4. If loop completes without finding divisor → prime (output 1).

FSMD State Diagram:

```
IDLE —(go=1)→ INIT ————————→ CHECK —(d*d>N)————————————→ PRIME
              (d=2,            |                            (out=1)
               result=1)       | d*d≤N
                               ▼
                             MOD —(N mod d=0)————————————→ NOT_PRIME
                              |                             (out=0)
                              | N mod d≠0
                              ▼
                            INC_D —→ CHECK
                           (d=d+1)
```

Datapath:

```
Registers: RN (input), Rd (divisor), Rresult (1=prime)
Units:
  Multiplier: Rsq ← Rd × Rd       (to check d²≤N)
  Comparator: gt_flag ← (Rsq > RN)
  Modulo:     Rmod ← RN mod Rd
  Zero-check: zero_flag ← (Rmod = 0)
  Incrementer: Rd ← Rd + 1

Example: N=7
  d=2: 4≤7, 7 mod 2=1 ≠0 → d=3
  d=3: 9>7 → loop exits → PRIME, output=1 ✓

Example: N=9
  d=2: 4≤9, 9 mod 2=1 ≠0 → d=3
  d=3: 9≤9, 9 mod 3=0 → NOT_PRIME, output=0 ✓
```

G1.8 What are the typical characteristics of an embedded system? Explain the various purposes and applications of embedded systems with examples.	**2073 Magh (Back) | 2072 Ashwin (Back) | 2076 Baisakh (Regular) | 2070 Magh (Back) [4]**

ANSWER

Characteristics of Embedded Systems:

- Single-functioned: Executes a dedicated program repeatedly (e.g., a washing machine controller only runs the wash cycle algorithm).
- Tightly constrained: Must meet strict design metrics — limited power (battery devices), small size (wearables), low cost (consumer goods), real-time deadlines (airbag controller <10ms).
- Reactive and real-time: Continuously reacts to the environment. Hard real-time systems must meet deadlines without exception (e.g., ABS brakes); soft real-time may tolerate occasional misses (e.g., video player).
- Microprocessor/microcontroller-based: Contains a processor, memory, and I/O on one or few chips.
- Connected to hardware: Interfaces with sensors (ADC input) and actuators (PWM/DAC output) directly.
- Reliability and safety-critical: Many embedded systems control safety-critical functions (medical devices, automotive) — must be highly reliable with fault tolerance.

Purposes and Applications:

Domain	Application	Embedded System Role
Consumer Electronics	Digital camera	Controls CCD/CMOS sensor, JPEG compression, LCD display, flash storage

Automotive	ABS brakes	Reads wheel speed sensors, computes slip ratio, modulates brake pressure in <10ms
Medical	Pacemaker	Monitors heart rhythm, delivers precise electrical pulses; ultra-low power
Communications	WiFi router	Packet routing, encryption/decryption, protocol stack — dedicated network processor
Industrial	PLC (CNC machine)	Reads encoder positions, runs motion control algorithms, drives servo motors
Home Appliances	Washing machine	Controls water level, motor speed, heating element based on user program
Aerospace	Flight control system	Fly-by-wire: sensor fusion, PID control, redundant fault-tolerant architecture

Chapter 2 — Additional Gap Topics

G2.3 Draw the basic architecture of a general-purpose processor and explain the programmer's view. Describe the criteria for selecting a processor for an embedded system.	**2082 Kartik (Back) | 2081 Chaitra (Regular) | 2076 Baisakh (Regular) | 2072 Ashwin (Back) [2+6]**

ANSWER

General-Purpose Processor Architecture:

```
┌──────────────────────────────────────────────────────────────┐
│                   General-Purpose Processor                  │
│                                                              │
│  ┌──────────┐   ┌──────────────────────────────────────┐    │
│  │   PC     │──►│         Instruction Memory (ROM)     │    │
│  └──────────┘   └──────────────────────────────────────┘    │
│                                  │  Instruction bus          │
│  ┌──────────────────────────────▼──────────────────────┐    │
│  │                    CONTROLLER                        │    │
│  │  IR → Decoder → State machine → Control signals      │    │
│  └──────────────────────────────┬──────────────────────┘    │
│                Control signals  │                            │
│  ┌──────────────────────────────▼──────────────────────┐    │
│  │                    DATAPATH                          │    │
│  │  ┌──────────┐   ┌──────┐   ┌──────────────────┐     │    │
│  │  │ Register │◄►│ ALU  │◄►│ Data Memory (RAM) │     │    │
│  │  │ File     │   │      │   │ MAR, MDR          │     │    │
│  │  │ R0-R31   │   └──────┘   └──────────────────┘     │    │
│  │  └──────────┘                                        │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

Programmer's View (ISA Level):

The programmer sees only the Instruction Set Architecture (ISA) — an abstract model that hides hardware details. It defines what the programmer needs to know to write correct programs.

- Registers: Number, size, and types of programmer-visible registers (e.g., R0–R15 in ARM, 32 GPRs in RISC-V, accumulator in 8051).
- Instruction set: Set of all valid opcodes, their operand types, encoding, and semantics (ADD, MOV, LOAD, STORE, BRANCH, etc.).
- Addressing modes: How memory operands are specified — immediate, register, direct, indirect, indexed, relative.
- Memory model: Address space size, byte/word ordering (big-endian vs little-endian), alignment requirements.
- I/O model: Memory-mapped I/O or port-based I/O; how peripherals are accessed from software.
- Exception/interrupt model: How hardware exceptions and interrupts are handled, ISR entry/exit mechanism.

Criteria for Selecting a Processor for Embedded Systems:

Criterion	Consideration
Performance	Required MIPS, clock speed, pipeline depth, hardware multiply/divide for real-time deadlines
Power consumption	Battery life target; static leakage + dynamic power; sleep modes available
Cost	Unit price (high volume = cost critical); NRE cost of development tools
Memory	On-chip ROM/RAM size; need for external memory (cost, board space, power)
Peripherals	Required I/O: UART, SPI, I2C, ADC, PWM, timers — built-in saves external ICs
Development tools	Compiler, debugger, RTOS support, IDE quality, cost of toolchain
Vendor support	Documentation, application notes, community, longevity, supply chain
Existing code	If existing codebase for specific architecture — reduces development time

G2.4 Explain the concept of pipelining. Differentiate between Superscalar and VLIW (Very Long Instruction Word) processor architectures.	**2079 Jestha (Back) | 2081 Ashwin (Back) [3+5]**

ANSWER

Pipelining — Brief Review:

Pipelining overlaps execution of multiple instructions by dividing the instruction cycle into stages (IF, ID, EX, MEM, WB). Each stage handles a different instruction simultaneously — like an assembly line. Ideal speedup = N (number of stages), limited by hazards and pipeline stalls.

Property	Superscalar	VLIW
Definition	Multiple execution units; hardware dynamically issues multiple instructions per cycle	Compiler packs multiple operations into one very long instruction word; hardware executes them in parallel
Parallelism detection	Dynamic — hardware detects independent instructions at runtime	Static — compiler detects and schedules parallel operations at compile time
Instruction width	Standard (32-bit); hardware decides what can run in parallel	Very wide (128–1024 bits); each field drives one functional unit
Hardware complexity	Very high — out-of-order execution, dynamic scheduling, branch prediction	Lower — simple in-order hardware; compiler does the hard work
Compiler requirement	Standard compiler works (hardware handles scheduling)	Needs intelligent compiler with knowledge of hardware resources
Performance	Excellent for general workloads with unpredictable branches	Excellent for regular, loop-intensive code (DSP, multimedia)
Power	Higher (complex dynamic hardware)	Lower (simpler hardware)
Examples	Intel Core i9, AMD Ryzen, ARM Cortex-A76	TI TMS320C6x DSP, Intel Itanium (IA-64), Trimedia

Chapter 3 — Additional Gap Topics

G3.2 Explain ISA bus protocols. Describe the principle of pipelining in the context of software implementation and verification phases.	**2079 Chaitra (Regular) | 2081 Ashwin (Back) [4+4]**

ANSWER

ISA Bus Protocol:

ISA (Industry Standard Architecture) is a legacy 16-bit system bus standard (introduced with IBM PC/AT). It connects the processor to memory and peripherals on the system board.

- Bus width: 16-bit data bus, 24-bit address bus (16 MB address space).
- Transfer speed: 8 MHz bus clock, ~8 MB/s peak transfer rate (slow by modern standards).
- Master-slave protocol: CPU (master) controls all bus transactions. Peripherals (slaves) respond when their address range is selected.
- Bus cycle — read sequence: (1) CPU drives address on address bus. (2) CPU asserts IOR# (I/O Read) or MEMR# (Memory Read) control signal. (3) Peripheral decodes address, places data on data bus. (4) CPU samples data bus. (5) CPU deasserts control signals — cycle complete.

- Bus cycle — write sequence: (1) CPU drives address. (2) CPU drives data onto data bus. (3) CPU asserts IOW# (I/O Write) or MEMW#. (4) Peripheral latches data. (5) CPU deasserts — cycle ends.
- Wait states: Slow peripherals assert CHRDY# (Channel Ready) LOW to insert wait states — CPU holds bus until peripheral is ready.
- DMA: ISA supports bus mastering DMA — peripheral requests bus, takes over address/data bus, transfers data to memory without CPU intervention.

Pipeline Principle (Processor Level):

Pipelining in processor design overlaps the 5 stages (IF, ID, EX, MEM, WB) so that while instruction I1 is in EX, I2 is in ID and I3 is in IF — all simultaneously. This achieves approximately 1 instruction completed per clock cycle in steady state (CPI ≈ 1 for ideal pipeline).

```
Pipeline stages executing concurrently:
  Cycle 5: I1-WB | I2-MEM | I3-EX | I4-ID | I5-IF
  → 5 instructions in-flight simultaneously
  → 1 instruction retires per cycle (vs 5 cycles for single-cycle
processor)
```

Chapter 5 — Additional Gap Topics

G5.2 Explain interrupt-driven I/O using fixed ISR location (non-vectored interrupt). Describe the major steps to be followed in interrupt processing by a processor.	**2079 Ashwin (Back) | 2079 Baisakh (Regular) [4+4]**

ANSWER

Interrupt-Driven I/O — Fixed ISR Location (Non-Vectored):

In a non-vectored interrupt system, all interrupt sources share a single fixed ISR address in memory. The processor always jumps to the same address when any interrupt occurs, regardless of source. The ISR must then poll hardware registers to determine which device caused the interrupt.

```
                 CPU                        Peripheral
                  |                              |
   Running main   |                              |  I/O complete
   program        |          IRQ ←---------------|
                  |                              |
   Save context ←-|                              |
   (push PC,      |                              |
    push flags)   |                              |
                  |                              |
   Jump to fixed -|                              |
   ISR address    |                              |
  (e.g., 0003H)   |                              |
```

```
                    |                              |
  Poll INT flags—┤  ← read STATUS register      |
  (which device?)|                              |
                    |                              |
  Service device ┤                              |
  (read/write      |                              |
   data)           |                              |
                    |                              |
  Clear INT flag ┤                              |
  Restore context|                              |
  (pop, RETI)      |                              |
                    |                              |
  Resume main    —┤                              |
```

Steps in Interrupt Processing:

1. Interrupt request: Peripheral asserts IRQ line when I/O is complete or data is ready.
2. Interrupt detection: CPU checks for pending interrupts at the end of each instruction cycle (not during instruction execution).
3. Interrupt acknowledgement: CPU asserts INTA (Interrupt Acknowledge) signal, peripheral lowers its IRQ.
4. Context save: CPU pushes Program Counter (PC/return address) and status flags (PSW/SREG) onto the stack — saving the interrupted task's state.
5. Vector/jump: CPU loads PC with fixed ISR address (e.g., 0003H for 8051 INT0). For non-vectored: single fixed address for all interrupts.
6. ISR execution: ISR polls status registers to identify interrupt source; reads/writes data to/from peripheral buffer; clears interrupt flag.
7. Context restore: ISR executes RETI (Return from Interrupt); CPU pops PC and flags from stack.
8. Resume: CPU continues executing main program from the exact point where it was interrupted.

G5.3 Design an interface circuit of a microprocessor system with 16-bit address bus interfacing 2 RAMs and 2 ROMs of 8 bits each. Explain the address decoding scheme.	**2073 Magh (Back) [8]**

ANSWER

Given: 16-bit address bus (64KB space); 2 RAMs + 2 ROMs, each 8-bit data, assume 4KB each.

```
Address space allocation (assuming 4KB each = 2^12 = 4096 locations):

  ROM1: 0000H – 0FFFH  (A15-A12 = 0000)
  ROM2: 1000H – 1FFFH  (A15-A12 = 0001)
  RAM1: 2000H – 2FFFH  (A15-A12 = 0010)
```

```
  RAM2: 3000H – 3FFFH  (A15-A12 = 0011)

Address bus connections:
  A11-A0 (12 bits) ──────────────────────► All chips (local address)
  A15-A12 (4 bits) ──►  2-to-4 Decoder  ─► CS0 → ROM1
                                          ─► CS1 → ROM2
                                          ─► CS2 → RAM1
                                          ─► CS3 → RAM2

Circuit block diagram:
  ┌──────────┐  A15-A12 ──►┌──────────────┐
  │   CPU    │             │ 4-bit decoder │──CS0──► ROM1 (4KB)
  │ (16-bit  │             │  (74LS139 or  │──CS1──► ROM2 (4KB)
  │  addr)   │             │   logic gates)│──CS2──► RAM1 (4KB)
  └──────────┘             └──────────────┘──CS3──► RAM2 (4KB)
       │                                              │
  A11-A0 ──────────────────────────────────────────► All chips (A11-A0)
  D7-D0  ◄──────────────────────────────────────────► Data bus (shared)
  RD#, WR# ────────────────────────────────────────► OE#, WE# on all chips

Note: OE# (Output Enable) connected to RD# signal from CPU
      WE# (Write Enable) connected to WR# signal from CPU
      ROM chips: WE# tied permanently HIGH (read-only)
```

Chapter 6 — Additional Gap Topics

G6.3 What is context switching? Explain the detailed steps involved in context switching. When is context switching necessary? Solve: P1–P4 with burst times 7,4,12,3ms entering together; P5 with burst 4ms enters after 2ms. Use Preemptive SJF (SRTF). Calculate WT, TAT, AWT, ATAT.	**2080 Chaitra (Back)** **[2+4+6]**

ANSWER

Context Switching:

A context switch is the process of saving the complete execution state (context) of the currently running process/task and restoring the saved context of the next process to run. It allows multiple processes to share a single CPU.

Situations When Context Switching is Necessary:

- Preemption by higher-priority task: In preemptive scheduling, a higher-priority process becomes ready — current process is preempted.
- Timer interrupt (time-slice expiry): In Round-Robin, the time quantum expires — scheduler switches to next process.
- I/O wait: Running process issues an I/O request and blocks — CPU switches to another ready process.

- Process termination: Running process completes and exits — scheduler picks next process.
- Synchronization: Process waits on a mutex/semaphore — blocked, CPU given to another process.

Steps Involved in Context Switching:

1. Interrupt/signal received: Timer interrupt or system call triggers scheduler.
2. Save CPU registers: All register values (PC, SP, PSW, general-purpose registers R0-R15) of current process saved to its Process Control Block (PCB) in memory.
3. Save stack pointer: Current process stack pointer saved so stack can be restored.
4. Update process state: Current process state changed from RUNNING to READY (preemption) or BLOCKED (I/O wait).
5. Run scheduler: Scheduling algorithm selects next process from READY queue.
6. Restore context: New process's register values loaded from its PCB into CPU registers.
7. Update memory maps: If processes use different address spaces, MMU (Memory Management Unit) page tables updated.
8. Resume execution: CPU jumps to saved PC of new process — it continues exactly where it was paused.

SRTF (Preemptive SJF) Numerical:

Process	Arrival	Burst
P1	0	7
P2	0	4
P3	0	12
P4	0	3
P5	2	4

```
SRTF: always run process with shortest REMAINING time.

t=0: Ready={P1(7),P2(4),P3(12),P4(3)} → shortest=P4(3) → run P4
t=2: P5 arrives with burst=4. Check remaining:
P4(1),P1(7),P2(4),P3(12),P5(4)
     Shortest=P4(1) → continue P4
t=3: P4 done. Ready={P1(7),P2(4),P3(12),P5(4)} → shortest=P2=P5 (tie, take
P2)
t=7: P2 done. Ready={P1(7),P3(12),P5(4)} → shortest=P5(4) → run P5
t=11: P5 done. Ready={P1(7),P3(12)} → shortest=P1(7) → run P1
t=18: P1 done. Ready={P3(12)} → run P3
t=30: P3 done.
```

```
Gantt: |P4:0-3|P2:3-7|P5:7-11|P1:11-18|P3:18-30|

Completion times: P1=18, P2=7, P3=30, P4=3, P5=11
Arrival times:    P1=0,  P2=0, P3=0,  P4=0, P5=2

TAT = Completion - Arrival:
  P1: 18-0=18  P2: 7-0=7  P3: 30-0=30  P4: 3-0=3  P5: 11-2=9

WT = TAT - Burst:
  P1: 18-7=11  P2: 7-4=3  P3: 30-12=18  P4: 3-3=0  P5: 9-4=5

AWT  = (11+3+18+0+5)/5 = 37/5 = 7.4 ms
ATAT = (18+7+30+3+9)/5 = 67/5 = 13.4 ms
```

G6.4 Differentiate between GPOS and RTOS. Explain process management. Three processes P1,P2,P3 (burst=10,5,7ms; priority=0,2,3 where 0=highest) enter together; P4 (burst=6ms, priority=1) enters after 6ms of P1. Non-preemptive priority scheduling. Calculate WT, TAT, AWT, ATAT.	**2078 Poush (Back) [3+3+6]**

ANSWER

Property	GPOS (General Purpose OS)	RTOS (Real-Time OS)
Primary goal	Maximum throughput and user responsiveness	Meet strict timing deadlines (deterministic)
Scheduling	Fair sharing (time-sliced RR, CFS)	Priority-based preemptive; rate-monotonic
Context switch time	Not bounded (may be ms)	Bounded and minimal (µs range)
Interrupt latency	Variable, not guaranteed	Deterministic, low and bounded
Memory management	Virtual memory, demand paging	Often no VM; fixed/static allocation
Examples	Windows, Linux, macOS	FreeRTOS, VxWorks, µC/OS-II, RTX

Process Management in RTOS:

- Process/Task states: NEW → READY → RUNNING → BLOCKED → TERMINATED. RTOS kernel manages transitions.
- PCB (Process Control Block): Data structure storing process state — registers, PC, SP, priority, stack, I/O status. One PCB per process.
- Ready queue: Ordered list of READY processes, sorted by priority. Scheduler picks highest priority.
- Task creation/deletion: RTOS API calls (osTaskCreate, osTaskDelete) to manage task lifecycle.

- Scheduling: Preemptive priority — highest priority READY task always runs. If equal priority, Round-Robin among them.

Non-Preemptive Priority Scheduling Numerical:

Process	Arrival	Burst	Priority (0=highest)
P1	0	10	0
P2	0	5	2
P3	0	7	3
P4	6	6	1

```
Non-preemptive: once a process starts, it runs to completion.

t=0: Ready={P1(pr=0),P2(pr=2),P3(pr=3)} → highest=P1(pr=0) → run P1
t=6: P4 arrives (pr=1) during P1's execution → queued but NO preemption
t=10: P1 done. Ready={P2(pr=2),P3(pr=3),P4(pr=1)} → highest=P4(pr=1)
t=16: P4 done. Ready={P2(pr=2),P3(pr=3)} → highest=P2(pr=2)
t=21: P2 done. Ready={P3(pr=3)} → P3 runs
t=28: P3 done.

Gantt: |P1:0-10|P4:10-16|P2:16-21|P3:21-28|

Completion: P1=10, P2=21, P3=28, P4=16
Arrival:    P1=0,  P2=0,  P3=0,  P4=6

TAT = Completion - Arrival:
  P1:10-0=10  P2:21-0=21  P3:28-0=28  P4:16-6=10

WT = TAT - Burst:
  P1:10-10=0  P2:21-5=16  P3:28-7=21  P4:10-6=4

AWT  = (0+16+21+4)/4 = 41/4 = 10.25 ms
ATAT = (10+21+28+10)/4 = 69/4 = 17.25 ms
```

Chapter 8 — Additional Gap Topics

G8.2 Briefly explain Semi-Custom (ASIC) IC technology and write about its types. Explain the design steps for Gate Array and Standard Cell semi-custom technologies.	**2076 Baisakh (Regular) | 2078 Poush (Back) [3+5]**

ANSWER

Semi-Custom IC Technology:

Semi-custom IC technology provides a middle ground between full-custom (complete manual layout) and PLD (programmable). The manufacturer pre-designs and fabricates certain layers of the chip; the designer customizes only the remaining layers (interconnect/personalization layers) to implement the desired logic. This significantly reduces NRE (Non-Recurring Engineering) cost and time-to-market compared to full-custom.

Types of Semi-Custom IC Technology:

1. Gate-Array (GA): Wafer is pre-fabricated up to the metal interconnect layers. It contains a regular array of unconnected transistor pairs (gates) covering the entire chip. The designer only adds the metal interconnect layers to implement the desired logic. Since all base layers are identical, wafers can be pre-manufactured in volume and customized quickly.

```
Gate-Array structure:
  ┌──────────────────────────────────────────┐
  │  I/O cells around perimeter              │
  │  ┌──────────────────────────────────────┐│
  │  │  Array of pre-fabricated gate cells ||
  │  │  (transistors without interconnect) ||
  │  │  ┌──┐┌──┐┌──┐┌──┐┌──┐┌──┐┌──┐┌──┐  ||
  │  │  |GA||GA||GA||GA||GA||GA||GA||GA|  ||
  │  │  └──┘└──┘└──┘└──┘└──┘└──┘└──┘└──┘  ||
  │  │  [repeated rows of gate cells]      ||
  │  └──────────────────────────────────────┘│
  └──────────────────────────────────────────┘
  Designer adds: Metal 1, Metal 2 layers (routing/personalization only)
```

2. Standard Cell (SC): A library of pre-designed, pre-characterized logic cells (AND, OR, NAND, NOR, DFF, MUX, etc.) is used. The designer synthesizes RTL code, maps to standard cells, and the P&R (place and route) tool arranges them in rows and connects them with routing channels between rows. All layers are custom-fabricated for each design.

```
Standard Cell layout structure:
  ┌──────────────────────────────────────────┐
  │  Power rail VDD ──────────────────────── │
  │  [Row of standard cells: INV NAND NOR DFF] │
  │  [Routing channel (interconnect wires)]    │
  │  [Row of standard cells: AND OR XOR MUX]   │
  │  [Routing channel]                         │
  │  [Row of standard cells: ...]              │
  │  Power rail GND ──────────────────────── │
  └──────────────────────────────────────────┘
```

Property	Gate Array	Standard Cell
Pre-fabrication	All layers except metal	None — all layers custom
Cell flexibility	Fixed gate primitives only	Rich library (100s of cells)
Area efficiency	Lower (unused gates wasted)	Higher (cells placed optimally)

NRE cost	Low (only metal masks)	Medium (all masks)
Turnaround time	Very fast (days–weeks)	Weeks–months
Design flow	Simpler	Full RTL→synthesis→P&R flow

Chapter 9 — Additional Gap Topics

G9.4 Draw the minimum configuration circuit for the 8051 microcontroller to operate. Show the connection of an LED at P1.7 and a switch at P1.1. Using assembly language, generate a pulse of 75% duty cycle at P1.7 when the switch at P1.1 is ON.	**2074 Bhadra (Regular) [5+3]**

ANSWER

8051 Minimum Configuration Circuit Components:

- Power supply: VCC (+5V) connected to pin 40 (VCC) and pin 31 (EA#). GND to pin 20 (GND).
- Crystal oscillator: 11.0592 MHz crystal between XTAL1 (pin 19) and XTAL2 (pin 18), each with 30pF capacitor to GND.
- Reset circuit: 10µF capacitor + 10kΩ resistor from RST (pin 9) to VCC — power-on reset. Optional push-button for manual reset.
- EA# pin (pin 31): Connected to VCC (+5V) — selects internal program memory (4KB internal ROM). If LOW, external program memory is used.
- LED at P1.7: LED + 330Ω current-limiting resistor connected from P1.7 (pin 8) to GND. LED ON when P1.7 = LOW (active LOW drive, as 8051 sinks current).
- Switch at P1.1: Switch connected between P1.1 (pin 2) and GND. When open: P1.1 = HIGH (internal pull-up). When closed: P1.1 = LOW.

```
Minimum Configuration Diagram:

  VCC (+5V)
   │  ├── Pin 40 (VCC)
   │  ├── Pin 31 (EA#)      ← selects internal ROM
   │  │
   │  └── 10µF ──┬── Pin 9 (RST)   ← power-on reset
                 └── 10kΩ ── GND

  XTAL 11.0592MHz:
    Pin 19 (XTAL1) ──┤crystal├── Pin 18 (XTAL2)
    Pin 19 ── 30pF ── GND
    Pin 18 ── 30pF ── GND

  Pin 20 (GND) ── GND

  LED circuit (P1.7, pin 8):
```

```
    Pin 8 (P1.7) ── 330Ω ── Anode –[LED]– Cathode ── GND
    LED ON when P1.7=0 (8051 sinks current, active LOW)

  Switch circuit (P1.1, pin 2):
    Pin 2 (P1.1) ──[SW]── GND
    Switch open:  P1.1=1 (internal pull-up)
    Switch closed: P1.1=0
```

```
;──────────────────────────────────────────────────────
; 75% duty cycle at P1.7 when switch P1.1=0 (switch ON)
; Period=4 units; HIGH=3 units, LOW=1 unit → 75% duty
;──────────────────────────────────────────────────────
        ORG   0000H
START:  MOV   P1, #0FFH        ; configure P1 as input
CHECK:  JB    P1.1, CHECK      ; loop while switch OFF (P1.1=1)

; Switch ON (P1.1=0): generate 75% duty pulse at P1.7
PULSE:  CLR   P1.7             ; P1.7 LOW → LED ON
        ACALL DLY              ; LOW period (1 unit = 25% of cycle)
        SETB  P1.7             ; P1.7 HIGH → LED OFF
        ACALL DLY              ; HIGH period
        ACALL DLY              ; HIGH period
        ACALL DLY              ; HIGH period (3 units = 75%)
        JNB   P1.1, PULSE      ; repeat while switch ON
        SJMP  CHECK

DLY:    MOV   R0, #250
D1:     NOP
        DJNZ  R0, D1
        RET
        END
```

G9.5 Explain PORT 3 of 8051 microcontroller. Using 8051 instructions, control the rate of blink of an LED at P1.1 by two switches at P2.1 and P2.2 (one to increase, one to decrease the rate).	**2078 Chaitra (Back) | 2076 Baisakh (Regular) [2+6]**

ANSWER

PORT 3 of 8051 — Alternate Functions:

Port 3 is an 8-bit quasi-bidirectional I/O port. Each pin has an important alternate (second) function used for communication, interrupts, and timers. The alternate function takes priority when enabled.

Pin	Port 3 Bit	Alternate Function	Description
10	P3.0 (RXD)	Serial receive	Receive data for UART serial communication
11	P3.1 (TXD)	Serial transmit	Transmit data for UART serial communication

12	P3.2 (/INT0)	External interrupt 0	Active-LOW external interrupt input
13	P3.3 (/INT1)	External interrupt 1	Active-LOW external interrupt input
14	P3.4 (T0)	Timer 0 external input	External clock/count input for Timer 0
15	P3.5 (T1)	Timer 1 external input	External clock/count input for Timer 1
16	P3.6 (/WR)	External write strobe	Write strobe for external data memory
17	P3.7 (/RD)	External read strobe	Read strobe for external data memory

```
;————————————————————————————————————————————————
; LED blink rate control using two switches
; P2.1: increase blink rate (decrease delay)
; P2.2: decrease blink rate (increase delay)
; LED at P1.1
; Delay stored in R7 (range: 10H to FFH)
;————————————————————————————————————————————————
        ORG   0000H
        MOV   P2,  #0FFH   ; P2 as input
        MOV   R7,  #80H    ; initial delay value (mid-range)

MAIN:   ; Toggle LED
        CPL   P1.1         ; toggle LED at P1.1

        ; Variable delay using R7
        MOV   R5,  R7      ; outer loop count from R7
DOUT:   MOV   R6,  #200
DIN:    NOP
        DJNZ  R6,  DIN
        DJNZ  R5,  DOUT

        ; Check P2.1 (increase rate = decrease delay)
        JB    P2.1, CHK2   ; skip if P2.1=1 (not pressed)
        MOV   A,   R7
        SUBB  A,   #10H    ; decrease delay by 16
        JC    CHK2         ; if underflow, don't change
        MOV   R7,  A       ; update delay

        ; Check P2.2 (decrease rate = increase delay)
CHK2:   JB    P2.2, MAIN   ; skip if P2.2=1 (not pressed)
        MOV   A,   R7
        ADD   A,   #10H    ; increase delay by 16
        JNC   MAIN         ; if overflow (>FFH), don't change
        MOV   R7,  A
        SJMP  MAIN
        END
```

Chapter 10 — Additional Gap Topics

G10.4 Explain COMPONENT declaration in VHDL with an example. Explain entity and architecture declaration with complete syntax. Write VHDL code for a Full Adder using two Half Adders as components (structural model).	**2075 Baisakh (Back) | 2079 Jestha (Back) | 2073 Magh (Back) [3+5]**

ANSWER

Entity Declaration:

The ENTITY defines the external interface of a VHDL design unit — it specifies the name of the module and all its input/output ports (the 'black box' view visible to other modules).

```
-- Entity declaration syntax:
entity ENTITY_NAME is
  [generic (                          -- optional: parameterized designs
    PARAM_NAME : type := default;
  );]
  port (                              -- port list: external signals
    port_name : direction type;       -- direction: in/out/inout/buffer
    ...
  );
end [entity] ENTITY_NAME;

-- Port directions:
--   in     : input only (read by architecture, driven externally)
--   out    : output only (written by architecture, read externally)
--   inout  : bidirectional (both read and written by architecture)
--   buffer : output that can also be read internally

-- Example:
entity Half_Adder is
  port(
    A, B : in  STD_LOGIC;
    S, C : out STD_LOGIC
  );
end Half_Adder;
```

Architecture Declaration:

```
-- Architecture declaration syntax:
architecture ARCH_NAME of ENTITY_NAME is
  -- Declarations section:
  component COMP_NAME ... end component;  -- component declarations
  signal SIGNAL_NAME : type;              -- internal signal declarations
  constant C : type := value;             -- constant declarations
begin
  -- Concurrent statements (execute simultaneously):
  -- 1. Concurrent signal assignment: Y <= A AND B;
```

```
  -- 2. Process statement:  process(A,B) begin ... end process;
  -- 3. Component instantiation: U1: COMP port map(...);
end [architecture] ARCH_NAME;
```

COMPONENT Declaration and Instantiation:

```
-- COMPONENT declaration (in architecture declarations section):
-- Tells VHDL about an existing entity that will be used as a sub-module.
component COMPONENT_NAME
  port(
    port_name : direction type;
    ...           -- must MATCH the ports of the corresponding entity
  );
end component;

-- Component instantiation (in architecture body):
INSTANCE_LABEL : COMPONENT_NAME
  port map(
    component_port => actual_signal,   -- named association (preferred)
    ...
  );

-- Note: Multiple instances of the same component can be created
-- (U1, U2, U3 etc.) — each is an independent copy in hardware.
```

```
-- ═══════════════════════════════════════════════════════
-- HALF ADDER (building block)
-- S = A XOR B;  C = A AND B
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Half_Adder is
  port(A, B : in STD_LOGIC; S, C : out STD_LOGIC);
end Half_Adder;

architecture Dataflow of Half_Adder is
begin
  S <= A XOR B;
  C <= A AND B;
end Dataflow;

-- ═══════════════════════════════════════════════════════
-- FULL ADDER using 2 Half Adders + 1 OR gate — Structural
-- Sum  = A XOR B XOR Cin
-- Cout = (A AND B) OR (B AND Cin) OR (A AND Cin)
-- Using: HA1(A,B)→S1,C1; HA2(S1,Cin)→Sum,C2; Cout=C1 OR C2
```

```
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Full_Adder_Structural is
  port(
    A, B, Cin : in  STD_LOGIC;
    Sum, Cout : out STD_LOGIC
  );
end Full_Adder_Structural;

architecture Structural of Full_Adder_Structural is

  -- Component declaration: Half Adder
  component Half_Adder
    port(A, B : in STD_LOGIC; S, C : out STD_LOGIC);
  end component;

  -- Internal signals (wires between components)
  signal S1 : STD_LOGIC;   -- sum output of first Half Adder
  signal C1 : STD_LOGIC;   -- carry output of first Half Adder
  signal C2 : STD_LOGIC;   -- carry output of second Half Adder

begin
  -- Instantiate first Half Adder: HA1(A,B) → S1, C1
  HA1: Half_Adder port map(A=>A, B=>B, S=>S1, C=>C1);

  -- Instantiate second Half Adder: HA2(S1,Cin) → Sum, C2
  HA2: Half_Adder port map(A=>S1, B=>Cin, S=>Sum, C=>C2);

  -- OR gate for final carry: Cout = C1 OR C2
  Cout <= C1 OR C2;

end Structural;

-- Verification trace: A=1, B=1, Cin=1
--   HA1: S1 = 1 XOR 1 = 0; C1 = 1 AND 1 = 1
--   HA2: Sum = 0 XOR 1 = 1; C2 = 0 AND 1 = 0
--   Cout = 1 OR 0 = 1
--   Result: Sum=1, Cout=1 → 1+1+1=3=11b ✓
```